# The Autonomous Duck Deployment Framework

## Designed Edition - Flow & Readability Update

A deterministic, end-to-end system for AI-assisted software development

**File-first memory**
Project context lives in local Markdown files, not fragile chat history.

**Architect then Builder**
Plan the what and why before any tool writes code.

**Dry run before mutation**
Every sprint pauses for review before implementation begins.

Framework Version 1.5 - ADDF - Quack!

# Contents

**1. Start Here: The Fastest Path to Using the Framework**

1.1 What this framework does

1.2 Minimum viable version

1.3 First 30-minute loop

1.4 Quick Start Guide

**2. The Framework in One Page**

**3. The 8-Step AI Project Deployment Lifecycle**

Step 1 - Research & Global Architecture

Step 2 - Initial Development (The Prototype)

Step 3 - Testing (Harness First)

Step 4 - Documentation (The Immutable Ledger)

Step 5 - Full-Scale Development

Step 6 - The Reflection Loop

Step 7 - Deployment & Maintenance

Step 8 - Resumption (The Pivot Protocol in Practice)

**4. Simple Example: Running One Tiny Project Through the 8 Steps**

4.1 Tiny DOMAIN.md example

4.2 Tiny Sprint 001 acceptance criteria example

**5. Core Philosophy**

5.1 Why vibe coding fails

5.2 The Rubber Duck Principle

5.3 Context Engineering Over Context Windows

5.4 The Central Rule

**6. Dual-Mode Orchestration Model**

6.1 The File System: The Sovereign Brain

6.2 Builder Permission Levels

6.3 Multi-CLI Protocol and Compute Agnosticism

6.4 Token Economics and Cost Management

**7. Setup and Scaffolding**

7.1 System Tooling

7.2 Initializing the Directory

7.3 Full Directory Structure

7.4 Populating Foundational Files
7.5 Context Loading Protocol
7.6 Security and Secrets Protocol
7.7 Git Branching Protocol
**8. Operating Modes**
8.1 Mode 1 - Quick Patch
8.2 Mode 2 - Feature Sprint
8.3 Mode 3 - Refactor / Migration
8.4 Mode 4 - Data Ingestion / Discovery
8.5 Mode 5 - Style Audit
8.6 Parallel Sprints
**9. Sprint Workflow: Step-by-Step**
**10. File Templates and Formats**
10.1 AGENTS.md
10.2 STATE.md
10.3 DOMAIN.md
10.4 DECISIONS.md (ADR Format)
10.5 COMMANDS.md
10.6 STYLE_GUIDE.md
10.7 SECURITY.md
10.8 GIT_STRATEGY.md
10.9 PROMPT_CHANGELOG.md
10.10 Sprint: requirements.md
10.11 Sprint: blueprint.md
10.12 Sprint: acceptance.md
10.13 Sprint: dry_run.md
10.14 Sprint: implementation_log.md
10.15 Sprint: human_review.md
10.16 Sprint: retrospective.md
10.17 Sprint: rollback_log.md
10.18 planning/backlog.md
**11. Domain Adaptations and Starter Template Library**
11.1 Domain Adaptations
11.2 Starter Template Library

**12. Comprehensive Prompt Catalog**

12.1 Persona Injector

12.2 Project Kickoff Interview Prompt

12.3 Resumption Prompt

12.4 Sprint Pack Generation Prompt

12.5 Builder Pre-Flight Dry Run Prompt

12.6 Architect Sanity Audit Prompt

12.7 Builder Implementation Authorization Prompt

12.8 Architect Implementation Review Prompt

12.9 Cross-Sprint Consistency Audit Prompt

**13. Quick Reference Checklists**

13.1 Before the first AI session

13.2 Before Builder writes code

13.3 Before committing a sprint

**14. Glossary**

# 1. Start Here: The Fastest Path to Using the Framework

This section is the practical entry point. The deeper theory appears later, but a new user should be able to begin with this page.

## 1.1 What this framework does

The Autonomous Duck Deployment Framework turns AI-assisted development into a repeatable file-based workflow. Instead of asking an AI to remember a long chat, you maintain a small set of local Markdown files that define the project, the current sprint, the rules, the risks, the commands, and the acceptance criteria. The AI only acts after those files exist.

## 1.2 Minimum viable version

For a first project, do not start with every file. Start with the minimum set, then add the rest as the project grows.

For a first project, the minimum viable file set is small. Add the rest only when the project needs them.

- AGENTS.md - Rules for Architect Mode, Builder Mode, permission levels, and hard constraints.
- DOMAIN.md - Business logic, entities, workflows, and what the project explicitly does not handle.
- STATE.md - The current goal, active sprint, blockers, next step, and recently changed files.
- COMMANDS.md - The run, test, build, lint, deploy, and rollback commands that prove the work.
- planning/sprints/sprint_001/ - The first sprint folder containing requirements.md, blueprint.md, acceptance.md, dry_run.md, implementation_log.md, human_review.md, retrospective.md, and rollback_log.md.

### 1.3 First 30-minute loop

1. Create the project folder and starter Markdown files.
2. Write a short DOMAIN.md: what the app does, the core entities, and what is out of scope.
3. Open Architect Mode in a capable LLM and generate a sprint pack: requirements.md, blueprint.md, and acceptance.md.
4. Open Builder Mode in a repository-aware coding tool and run a Level 0 dry run. The Builder writes or returns dry_run.md and then stops.
5. Review the dry run. If it matches the blueprint, authorize Level 1 implementation. After implementation, run the commands from COMMANDS.md, update STATE.md, and close the Builder thread.

### 1.4 Quick Start Guide

Follow this minimal operational loop to initialize your environment and execute your first feature sprint using any available LLM service.

1. Scaffold your workspace root. Launch your terminal environment and create the target tracking architecture.
2. Seed your domain context. Open DOMAIN.md and type a brief summary of the app mission, explicit business rules, and elements that remain out of scope.
3. Launch Architect Mode. Copy the full contents of AGENTS.md, STATE.md, and DOMAIN.md into a clean Architect session. Run the Sprint Pack Generation Prompt to produce requirements.md, blueprint.md, and acceptance.md.
4. Execute Builder Mode. Open a local repository-aware coding extension. Load the sprint files and run the Builder Pre-Flight Dry Run Prompt. Review dry_run.md. Then authorize: Dry run approved. Permission Level 1 authorized. Proceed with code generation.
5. Audit and reset. After the Builder finishes and local tests pass, paste implementation_log.md back into the Architect session for compliance review. Update tracking sheets, close the Builder thread, and start the next sprint in a clean window.

```
mkdir -p src docs planning/backlog planning/sprints/sprint_001
touch AGENTS.md STATE.md DOMAIN.md DECISIONS.md COMMANDS.md STYLE_GUIDE.md SECURITY.md RISKS.md QUESTIONS.md FILE_INVENTORY.md
```

# 2. The Framework in One Page

Think of the framework as a small operating system for AI-assisted development. The project brain lives in local Markdown files. The Architect plans the work from those files. The Builder implements only the approved plan. A dry run creates a pause before code changes happen, and every sprint ends by updating the files so the next session starts clean.

### The project brain

Local File Sovereignty means the important memory of the project lives in AGENTS.md, DOMAIN.md, STATE.md, DECISIONS.md, COMMANDS.md, and sprint folders. These files survive rate limits, model swaps, lost chats, and long breaks.

### The two roles

Architect Mode answers what and why. It turns raw goals into requirements, blueprints, and acceptance criteria. Builder Mode answers how. It writes source code only after the blueprint and dry run have been approved.

### The safety gate

The Dry Run Gate requires the Builder to report exactly what it intends to create or modify before changing files. That gives the human operator a chance to catch hidden scope creep, accidental refactors, dependency additions, or business-logic invention.

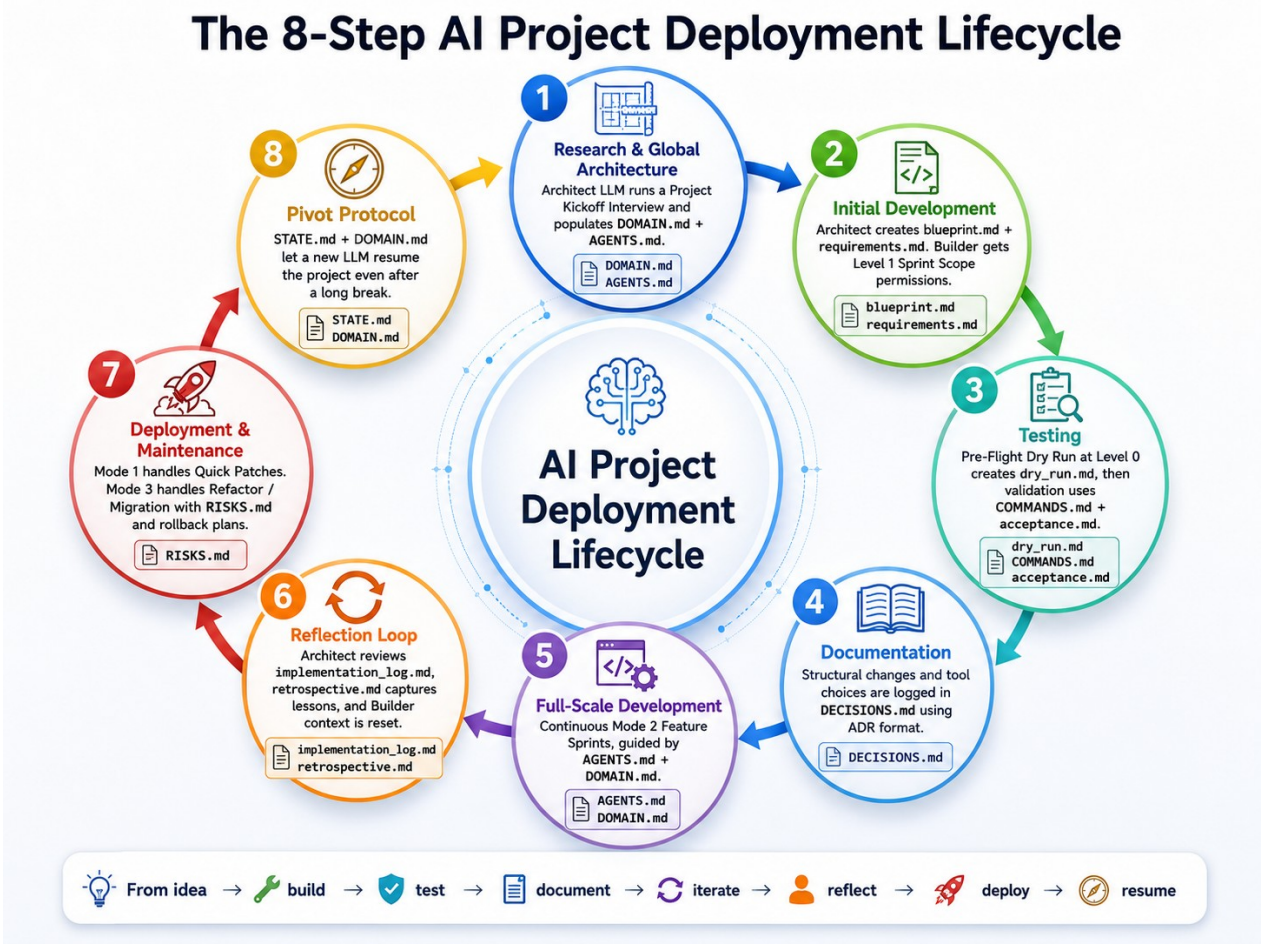
### The reset habit

At the end of a sprint, the implementation log, decisions, and current state are written back into the local files. The Builder chat can then be closed. The next sprint starts from clean project memory instead of a bloated conversation thread.

> **Central rule:** The question is never which AI does what. The question is what files must exist before any AI is allowed to act.

# 3. The 8-Step AI Project Deployment Lifecycle

Visual overview: the 8-step lifecycle moves from idea to build, test, document, iterate, reflect, deploy, and resume.




This is the central organizing spine of the framework. For a new project, the steps are sequential. After Step 7, loop back to Step 2 for the next feature cycle. Step 8 can happen at any point when returning to a paused project.

At a glance, the lifecycle moves from idea to architecture, then prototype, testing, documentation, feature expansion, reflection, deployment, and later resumption.

- Step 1: Research & Global Architecture - build the project brain with DOMAIN.md, AGENTS.md, STATE.md, DECISIONS.md, QUESTIONS.md, RISKS.md, and docs/architecture.md.
- Step 2: Initial Development - create the first working version using requirements.md and blueprint.md.
- Step 3: Testing - define acceptance.md and record validation output in implementation_log.md.
- Step 4: Documentation - update docs/data_model.md, docs/api.md, and DECISIONS.md when structural choices change.
- Step 5: Full-Scale Development - run back-to-back Feature Sprints from updated state and sprint folders.
- Step 6: Reflection Loop - convert mistakes into constraints through retrospective.md, AGENTS.md, and planning/backlog.md.

- Step 7: Deployment & Maintenance - ship safely with COMMANDS.md, RISKS.md, and rollback_log.md.
- Step 8: Resumption - return later by loading STATE.md, DOMAIN.md, and AGENTS.md.

## Step 1 - Research & Global Architecture

**The goal:** Build the project brain before writing a single line of code.

**Why this phase exists:** Every failure mode of AI-assisted development traces back to skipping this step. When the AI has no specification to work from, it invents one. The Step 1 output is the specification that every subsequent phase works from.

**The tool:** Architect LLM, with Claude CLI preferred when available.

**The action:** Open the Architect LLM and feed it raw ideas, feature wishlists, client emails, and rough notes. Run the Project Kickoff Interview prompt. The Architect asks clarifying questions and generates foundational files.

**Output files produced:** DOMAIN.md, STYLE_GUIDE.md, docs/architecture.md, STATE.md, DECISIONS.md, QUESTIONS.md, RISKS.md.

**The rule:** Do not proceed to Step 2 until DOMAIN.md contains real business definitions, SECURITY.md is populated, and there are no QUESTIONS.md items marked as sprint blockers.

## Step 2 - Initial Development (The Prototype)

**The goal:** Build the first working version of the core feature set under strict Architect control.

**The tool:** Architect LLM for blueprint generation; Builder agent for execution.

**The action:** Human reviews ledger files, Architect generates a Sprint Pack, human checks the Blueprint Review Gate, Builder performs a Level 0 Pre-Flight Dry Run, Architect audits, and Builder executes at Level 1 on branch sprint/001-[description].

**The rule:** The Builder generates only the files listed in the blueprint on the designated branch. No business logic is invented. No files outside the blueprint scope are touched.

## Step 3 - Testing (Harness First)

**The goal:** Establish a testing harness before functional code scales beyond the prototype.

**Why this phase matters:** The most effective AI development teams invert the vibe coding testing habit. Instead of building first and checking visually after, they build the evaluation framework before the AI writes implementation code.

**The action:** Before the Builder writes new functional code for a sprint, mandate that it writes the unit tests first. The acceptance.md checklist drives what must be proven before the sprint closes.

**The rule:** The Builder must run all commands listed in COMMANDS.md and paste terminal output directly into implementation_log.md. Visual inspection alone is not acceptance.

## Step 4 - Documentation (The Immutable Ledger)

**The goal:** Keep the architectural record current as the codebase grows.

**Why this phase matters:** Documentation debt is the silent killer of AI-assisted projects. If context files fall out of sync with code, the AI begins improvising again.

**The action:** After every sprint that introduces a structural change, command the Architect to update docs/data_model.md and docs/api.md.

**The rule:** Any change to the data model, API contract, auth system, or external dependencies requires a new DECISIONS.md entry in ADR format before the next sprint begins.

## Step 5 - Full-Scale Development

**The goal:** Execute the complete product build through back-to-back Feature Sprints.

**Why this phase matters:** At scale, improvisation bias compounds. A Builder that invents one database key in Sprint 3 may reference it in Sprint 7, Sprint 12, and Sprint 18.

**The action:** Execute Mode 2 Feature Sprints in sequence. Close the Builder chat completely between every sprint. Every third sprint, run the Cross-Sprint Consistency Audit to catch drift.

**The rule:** Each new Builder session begins by loading only the freshly updated STATE.md and the current sprint folder. The context reset between sprints is non-negotiable.

## Step 6 - The Reflection Loop

**The goal:** Turn one-time errors into permanent constraints, keep the context window clean, and maintain the backlog.

**Why this phase matters:** AI coding tools repeat mistakes unless explicitly told not to. retrospective.md and AGENTS.md convert failures into guardrails.

**The action:** Architect reviews log loops, human fills in retrospective metrics, updates constraints inside AGENTS.md, triages backlog items, and closes the Builder thread completely.

## Step 7 - Deployment & Maintenance

**The goal:** Move the product to production and manage it reliably over time.

**The action:** Minor fixes match Mode 1 Quick Patches. Heavy database migrations match Mode 3 Restructures.

**The rule:** Rollback commands must be present in COMMANDS.md before deployment. Fragile production assumptions are logged in RISKS.md with specific mitigation plans.

## Step 8 - Resumption (The Pivot Protocol in Practice)

**The goal:** Return to any paused project after days, weeks, or months in minutes.

**Why this phase matters:** Because project intelligence lives in Markdown files and not in AI memory, there is nothing to reconstruct. Load the key files and continue.

**The action:** Update STATE.md before session close. When resuming, upload STATE.md, DOMAIN.md, and AGENTS.md to the Architect LLM and run the Resumption Prompt.

**The rule:** STATE.md must be updated at the close of every session without exception.

# 4. Simple Example: Running One Tiny Project Through the 8 Steps

Example project: a tiny web app called Mini Task Tracker. It lets a user add tasks, mark them complete, and persist them locally. This example is intentionally small so the lifecycle is visible.

Here is how the Mini Task Tracker example moves through the lifecycle in practice.

#### Step 1 - Research & Global Architecture

Write the business rules before coding. DOMAIN.md states that a Task has a title, status, and createdAt value. It also states that user accounts, sharing, and cloud sync are out of scope.

#### Step 2 - Initial Development

Ask the Architect for Sprint 001. The output is a sprint pack: requirements.md says the user can add, list, and complete tasks; blueprint.md identifies TaskList, TaskForm, and local storage helpers.

#### Step 3 - Testing

Define success before building. acceptance.md states that an added task appears in the list, complete toggles status, and refreshing the page preserves tasks.

#### Step 4 - Documentation

Record structural decisions. DECISIONS.md notes that browser localStorage is used for version 1 and cloud sync is deferred.

#### Step 5 - Full-Scale Development

Run the next feature sprint. Sprint 002 might add filters for All, Active, and Completed. Sprint 003 might add editing for task titles.

#### Step 6 - Reflection Loop

Capture what broke. If the Builder forgot empty-title validation, retrospective.md records the miss and AGENTS.md gains a new constraint to validate empty inputs.

#### Step 7 - Deployment & Maintenance

Ship and patch safely. COMMANDS.md includes npm run build and rollback commands. A small typo fix later can run as a Mode 1 Quick Patch.

#### Step 8 - Resumption

Come back later. Load STATE.md, DOMAIN.md, and AGENTS.md, then ask the Architect to summarize the current state and propose the next sprint.

## 4.1 Tiny DOMAIN.md example

```
# DOMAIN.md Business Logic and Definitions
## Project Overview
Mini Task Tracker is a local-only task management app for a single user. It helps a user record small tasks and mark them complete.

## Core Entities
### Task
```

```
Definition: A single item the user wants to complete.
Key fields: id, title, status, createdAt, completedAt.
Business rules: A task title cannot be empty. A completed task can be reopened. The app stores tasks locally.
What it is NOT: A shared task, a cloud record, a calendar event, or a reminder.

## Out of Scope
- User accounts
- Multi-device sync
- Email notifications
- Team sharing
```

### 4.2 Tiny Sprint 001 acceptance criteria example

```
# Sprint 001 Acceptance Criteria
## Functional Criteria
- [ ] User can type a task title and add it.
- [ ] Newly added tasks appear in the list immediately.
- [ ] User can mark a task complete.
- [ ] Completed state remains after browser refresh.

## Technical Criteria
- [ ] No type errors.
- [ ] Build completes successfully.
- [ ] Test results are pasted into implementation_log.md.

## Verification Commands
1. npm run test
2. npm run build
```

# 5. Core Philosophy

## 5.1 Why vibe coding fails

Vibe coding is what happens when a developer describes an app idea to an AI tool, hits generate, eyeballs the output, and then asks for changes in the same chat thread indefinitely. It feels productive because output appears immediately. It fails predictably because of three structural weaknesses that compound over time.

#### Context bleed and memory bloat

A long chat thread fills with old code blocks, abandoned directions, and conversational filler. The AI loses track of decisions made many messages ago, and regression bugs appear that cannot be explained.

#### The improvisation bias

When an AI builder lacks a strict specification, it invents. It creates database keys that do not match the schema, assumes permission levels that clash with business logic, and takes the path of least resistance.

### The missing anchor

In vibe coding, conversation history becomes the project memory. That memory is fragile, unsearchable, non-portable, and eventually destroyed by token limits or session loss.

## 5.2 The Rubber Duck Principle

The name comes from rubber duck debugging: explaining code out loud to an inanimate object forces the developer to structure thinking clearly enough to find the problem. This framework applies that principle to AI-assisted development. Before the AI acts, the human operator structures the problem clearly enough to explain it in writing. The Markdown files are that explanation.

> **A rubber duck cannot invent your business logic. Neither should your AI.**

## 5.3 Context Engineering Over Context Windows

Core principle: more context is not better context. Cleaner context is better context. Large context windows do not solve the vibe coding problem; they often extend it. Dumping an entire codebase and a long chat history into a prompt is still disorganized context. Context Engineering divides project rules, history, and specifications into clean, modular Markdown files. Each AI session receives only the files it needs for the current job.

## 5.4 The Central Rule

The question is never which AI does what. The question is what files must exist before any AI is allowed to act. If the required documentation does not exist before a session begins, the session does not begin. The AI role is to execute against a specification. Writing the specification is the human operator job. Chat windows are disposable. The files are permanent.

# 6. Dual-Mode Orchestration Model

The framework is anchored by a Dual-Mode Orchestration Model. Any capable Large Language Model can execute any task if the operator clearly shifts between two operational modes: Architect Mode and Builder Mode.

### Architect Mode

Architect Mode can run in any capable LLM, including Claude Pro, ChatGPT Plus, Gemini Advanced, a terminal interface, or a browser project sandbox. Its responsibility is to dictate the what and why: process definitions, map relational fields, mitigate structural code risk, resolve logical unknowns, and produce development blueprints.

Its outputs are requirements.md, blueprint.md, and acceptance.md. Its strict constraint is that it does not write codebase source files directly, does not read live runtime logs, and explicitly catalogs structural boundaries.

### Builder Mode

Builder Mode runs in a local repository-aware coding environment such as Cursor, Claude Code, Cline, or Codex. Its responsibility is to dictate the how: consume approved schematics, generate source files inside /src, execute local tests, and register environment updates.

Its outputs are source files, dry_run.md, implementation_log.md, and test output. Its strict constraint is that it may not invent unapproved business rules. It must execute a read-only dry run before changing files and must stop after the dry run until permission is granted.

## 6.1 The File System: The Sovereign Brain

The core brain of the framework is localized on your drive within an organized collection of Markdown tracking files. Because instructions survive independently of any model state, you can run an Architect session in Claude, hit a rate limit, copy the updated tracking files, drop them into ChatGPT or Gemini, and pick up without context degradation.

## 6.2 Builder Permission Levels

| Level | Name | Operational Execution Parameters |
|---|---|---|
| 0 | Read-Only | Structural analysis and file scanning only. Zero code mutation or file creation permitted. Used for pre-flight dry runs. |
| 1 | Sprint Scope | Standard starting default. The agent can modify or create only files explicitly cataloged in the active sprint blueprint block. |
| 2 | Approved Expansion | The agent can generate minor auxiliary helper modules if surfaced during the dry run and validated by the Architect pack. |
| 3 | Supervised Refactor | Authorized to mutate adjacent files outside the core blueprint path, provided it indexes every change inside logging modules. |
| 4 | Migration | Heavy schema adjustments or complete architecture overhauls. Requires explicit rollback logs before processing begins. |

## 6.3 Multi-CLI Protocol and Compute Agnosticism

Models are treated as interchangeable, temporary compute engines piped into local shell utilities. The developer commands the ecosystem by loading system configuration files into terminal arguments.

The framework is compute-agnostic: models are temporary engines that read the same local files.

- Claude CLI is useful for schema planning, blueprint assembly, and code-review checks. Example: cat AGENTS.md STATE.md DOMAIN.md | claude "Compile Sprint Pack NNN"

- Gemini CLI is useful for high-capacity ingestion during data parsing discovery gates. Example: cat messy_log.csv | gemini "Map data vectors per AGENTS.md rules"
- GPT CLI is useful for structural code style verification and rule compliance checks. Example: cat STYLE_GUIDE.md src/app.ts | gpt "Identify design standard fractures"

### 6.4 Token Economics and Cost Management

Keep runtime expenses efficient by parsing compact data payloads, splitting large inventory lists, and routing basic text translation tasks to smaller, highly optimized models such as GPT-4o-mini or Claude Haiku.

## 7. Setup and Scaffolding

### 7.1 System Tooling

Use simple tools at the start. Add terminal and CLI layers only when they actually improve the workflow.

- Obsidian - primary interface for managing Markdown files, Architect Packs, STATE.md, prompts, and the project brain.
- VSCode - primary interface for the Builder layer; AI coding extensions execute from here.
- WSL on Windows - Bash environment for scripts, git commands, and local servers.
- Git - every sprint ends with a clean commit on its branch before context reset.
- Claude CLI - Architect sessions via terminal.
- Gemini CLI - discovery and data ingestion sessions.
- GPT CLI - validation and style enforcement sessions.

### 7.2 Initializing the Directory

```
# Create primary project directory
mkdir -p /mnt/c/Users/Public/Projects/[PROJECT_NAME]
cd /mnt/c/Users/Public/Projects/[PROJECT_NAME]

# Initialize git and create branch structure
git init
git checkout -b main
git checkout -b dev

# Build structural scaffolding
mkdir -p src docs planning/meetings planning/sprints/sprint_001 _templates

# Generate root tracking files
touch AGENTS.md README.md STATE.md DECISIONS.md DOMAIN.md RISKS.md QUESTIONS.md FILE_INVENTORY.md
COMMANDS.md STYLE_GUIDE.md SECURITY.md GIT_STRATEGY.md PROMPT_CHANGELOG.md

# Generate documentation files
touch docs/architecture.md docs/data_model.md docs/api.md docs/permissions.md docs/validation.md
```

```
# Generate sprint templates
touch planning/backlog.md
touch planning/sprints/sprint_001/requirements.md planning/sprints/sprint_001/blueprint.md
planning/sprints/sprint_001/blueprint_feedback.md planning/sprints/sprint_001/acceptance.md
planning/sprints/sprint_001/dry_run.md planning/sprints/sprint_001/implementation_log.md
planning/sprints/sprint_001/human_review.md planning/sprints/sprint_001/retrospective.md
planning/sprints/sprint_001/rollback_log.md

# Create .gitignore
echo -e ".env
.env.*
*.key
*.pem
secrets/
" > .gitignore
```

## 7.3 Full Directory Structure

```
Project-Root/
├── AGENTS.md            # AI persona rules and permission levels
├── STATE.md             # Living roadmap: current sprint, status, blockers
├── DOMAIN.md            # Business logic, entities, and workflows
├── DECISIONS.md         # Immutable ADR ledger of design choices
├── FILE_INVENTORY.md    # Source asset manifest and data shapes
├── COMMANDS.md          # Run, build, test, deploy, and rollback commands
├── STYLE_GUIDE.md       # Naming conventions, UI rules, code standards
├── SECURITY.md          # Safe-to-share and never-share file lists
├── GIT_STRATEGY.md      # Branching model and commit rules
├── PROMPT_CHANGELOG.md  # Version history of all framework prompts
├── RISKS.md             # Known risks and mitigation strategies
├── QUESTIONS.md         # Unresolved questions blocking design
├── README.md            # Project overview
├── .gitignore           # Excludes secrets and env files
├── docs/
│   ├── architecture.md, data_model.md, api.md, permissions.md, validation.md
├── planning/
│   ├── backlog.md       # Future ideas outside current scope
│   └── sprints/
│       └── sprint_001/
│           ├── requirements.md, blueprint.md, acceptance.md
│           ├── blueprint_feedback.md
│           ├── dry_run.md
│           ├── implementation_log.md
│           ├── human_review.md
│           ├── rollback_log.md
│           └── retrospective.md
├── _templates/          # Starter templates for new projects
```

```
└── src/              # Execution codebase
```

## 7.4 Populating Foundational Files

1. SECURITY.md: Fill in safe-to-share and never-share lists before anything else. This file sets boundaries for every later session.
2. AGENTS.md: Paste the permission rules and persona instructions. This file is loaded in every session and is the highest-priority instruction set.
3. DOMAIN.md: Write business definitions. Every entity, workflow, and rule not defined here is a rule the AI may invent.
4. COMMANDS.md: Document every command needed to run, test, build, deploy, and roll back before writing code.
5. GIT_STRATEGY.md: Fill in the branching model for this project.
6. STATE.md: Initialize using the strict short format. Set status to Discovery Phase.
7. DECISIONS.md: Create an empty ledger with the ADR header ready.
8. QUESTIONS.md and RISKS.md: Create empty ledgers with headers. Add known risks or questions from the outset.

## 7.5 Context Loading Protocol

Loading the wrong files wastes tokens and introduces noise. Load only the files required for the session type.

| Session Type | Load These Files | Do Not Load | Security Check |
|---|---|---|---|
| Architect Planning | AGENTS.md, STATE.md, DOMAIN.md, DECISIONS.md, QUESTIONS.md, RISKS.md | Full source code, old sprint folders | No - these files should never contain secrets |
| Builder Implementation | AGENTS.md, STATE.md, sprint requirements + blueprint + acceptance, COMMANDS.md, and only source files the sprint touches | DECISIONS.md, old sprints, unrelated files | No - but verify COMMANDS.md has no inline credentials |
| Audit Verification | acceptance.md, dry_run.md, implementation_log.md, test output | Source files, DOMAIN.md | No |
| Discovery Data Ingestion | AGENTS.md, FILE_INVENTORY.md, raw source documents | Sprint files, source code | Yes - redact all PII and credentials before loading |
| Validation Style Check | AGENTS.md, | DOMAIN.md, sprint files | No |

| | |
|---|---|
| | STYLE_GUIDE.md, files being validated |

### 7.6 Security and Secrets Protocol

**Highest-priority rule:** API keys, database connection strings, OAuth secrets, private keys, .env contents, real user PII, internal IPs/private domains, and payment information must never appear in files uploaded to an LLM session.

- Redaction rule for Mode 4 Discovery: replace sensitive values with REDACTED or synthetic placeholders before the file leaves your machine.
- Safe-to-share list: framework Markdown files are safe to upload only if written according to the protocol and free of secrets.
- The .gitignore file must include .env, .env.*, *.key, *.pem, and secrets/ directories before the first commit.
- SECURITY.md is the project-specific record of what is safe and not safe for this project. Update it when new credential types are introduced.

### 7.7 Git Branching Protocol

| Branch | Purpose | Who Commits |
|---|---|---|
| main | Production-ready code only. | Human operator, after Architect approval. |
| dev | Integration branch. All sprint branches merge here first. | Human operator, after human_review.md passes. |
| sprint/NNN-short-description | One branch per sprint. | Builder agent. |

- The Builder always works on a named sprint/NNN branch. This is stated explicitly at the start of every Builder session.
- Mode 1 Quick Patch is the only mode that may work directly on dev, given its scope constraint of under 10 lines with no new business logic.
- Mode 3 Refactor/Migration requires a dedicated refactor/description branch with an explicit merge plan in blueprint.md.

## 8. Operating Modes

Operating Modes define how to handle individual tasks within the larger lifecycle. Choose your mode before opening any AI session.

| Situation | Mode | Lifecycle Phase |
|---|---|---|
| Starting a brand-new project | Phase 1 Kickoff (pre-mode) | Phase 1 |
| Building the first working feature | Mode 2 - Feature Sprint | Phase 2 |

| | | |
|---|---|---|
| Adding a new feature | Mode 2 - Feature Sprint | Phase 5 |
| Fixing a bug or small config change | Mode 1 - Quick Patch | Phase 7 |
| Upgrading a dependency or migrating data | Mode 3 - Refactor/Migration | Phase 7 |
| Parsing raw data files or mapping a new API | Mode 4 - Data Ingestion | Phase 1 or 5 |
| Enforcing code style across new files | Mode 5 - Style Audit | Phase 5 or 7 |
| Running two non-overlapping features at once | Mode 2x2 - Parallel Sprints | Phase 5 |
| Returning to a paused project | Phase 8 Resumption (pre-mode) | Phase 8 |

## 8.1 Mode 1 - Quick Patch

**Use for:** Typo fixes, minor UI adjustments, single-function changes, renaming, or small config edits. Any change under approximately 10 lines that introduces no new business logic.

**Workflow / rule:** Open Builder at Level 1 permission on dev branch -> state the specific file and change -> Builder mutates file -> run verification from COMMANDS.md -> commit -> update STATE.md.

## 8.2 Mode 2 - Feature Sprint

**Use for:** New features, new routes, new components, new integrations, or any change that touches more than one file or introduces new business logic.

**Workflow / rule:** Pre-flight checklist: no open blocking QUESTIONS.md, no unresolved High/High RISKS.md, Architect loaded with AGENTS.md + STATE.md + DOMAIN.md, branch instantiated, sprint folder templates initialized.

## 8.3 Mode 3 - Refactor / Migration

**Use for:** Database migrations, major dependency upgrades, architectural restructuring, auth system changes, or anything that could break existing functionality.

**Workflow / rule:** Builder operates at Level 3 or Level 4, assigned explicitly. Architect reviews implementation_log.md before merge to dev or main. Retrospective is mandatory.

## 8.4 Mode 4 - Data Ingestion / Discovery

**Use for:** Parsing messy spreadsheets or PDFs, ingesting external APIs, mapping new data sources, or building ETL pipelines.

**Workflow / rule:** Security gate: all PII replaced with synthetic placeholders, API keys replaced with [REDACTED], and files reviewed against SECURITY.md never-share list.

### 8.5 Mode 5 - Style Audit

**Use for:** Enforcing STYLE_GUIDE.md conventions across new or modified files.

**Workflow / rule:** Recommended after any sprint that creates more than three new files. Mandatory before production deployment.

### 8.6 Parallel Sprints

**Use for:** Two or more feature sprints at the same time.

**Workflow / rule:** Permitted only when blueprints list zero overlapping files, each has its own branch/folder/Builder session, and Architect Sanity Audit confirms zero file overlap.

## 9. Sprint Workflow: Step-by-Step

The sprint workflow is the daily operating loop of the framework. Use it whenever a task is larger than a quick patch.

#### Step 1

Pre-Flight Gate: verify no blocking questions, no unresolved High/High risks, and sprint branch exists. Main artifact: QUESTIONS.md, RISKS.md, git branch.

#### Step 2

Prime the Architect: load ledgers and run Sprint Pack Generation Prompt. Main artifact: requirements.md, blueprint.md, acceptance.md.

#### Step 2B

Human Blueprint Review Gate: cross-examine assumptions and reject if needed. Main artifact: blueprint_feedback.md.

#### Step 3

Pre-Flight Dry Run: Builder runs Level 0 and stops. Main artifact: dry_run.md.

#### Step 4

Architect Sanity Audit: Architect checks hidden scope bleed and issues permission token. Main artifact: Permission approval.

#### Step 5

Dependency Approval Gate: new dependencies require a decision record. Main artifact: DECISIONS.md.

#### Step 6

Authorize and Build: Builder receives explicit permission and implements. Main artifact: src files, implementation_log.md.

#### Step 7

Run Verification: Builder runs all commands and records console output. Main artifact: COMMANDS.md, implementation_log.md.

#### Step 8

Architect Reviews Output: implementation_log.md is checked against acceptance.md. Main artifact: Architect review.

#### Step 9

Human Approval Gate: verify scope, tests, debt, decisions, and state. Main artifact: human_review.md.

#### Step 10

Commit, Update, Reset: commit branches, update state, complete retrospective, close Builder thread. Main artifact: git commit, STATE.md, retrospective.md.

# 10. File Templates and Formats

Copy these layouts into local Markdown files. The templates are intentionally simple so they are easy to maintain and easy for an AI session to parse.

## 10.1 AGENTS.md

```
# AGENTS.md System Operational Rules
## Architect Mode Constraints
When invoked in Architect Mode, you operate strictly at the systems specification layer.
1. Never write application source files or code blocks directly.
2. Enforce a structural Discovery Gate before approving implementation work. Interrogate code constraints and edge cases.
3. Your primary outputs are structured Markdown blueprints: requirements.md, blueprint.md, and acceptance.md.

## Builder Mode Constraints
When invoked in Builder Mode, you act as a localized implementation contractor.
1. Never invent database keys, entity behaviors, or business rules omitted from DOMAIN.md.
2. Operate strictly within the permission level allocated by the human operator for the active session.
3. You must execute an automated Read-Only pre-flight dry run and stop generation immediately to await human approval.

## Builder Permission Levels
- Level 0: Read-only. Structural scanning and dry run execution only. No code mutation permitted.
- Level 1: Standard Sprint Scope. Mutate only files explicitly logged in the active blueprint sheet.
- Level 2: Approved Expansion. Allowed to build secondary utility scripts highlighted in the dry run.
- Level 3: Supervised Refactor. Authorized to clean adjacent files, provided all edits are recorded in logs.
- Level 4: Migration. Heavy structural schema updates. Requires pre-written rollback procedures.
```

## 10.2 STATE.md

```
# STATE.md
## Current Goal
[One sentence: what are we building right now?]

## Active Sprint
```

```
Sprint [NUMBER]: [Brief description]
Branch: sprint/[NNN]-[description]

## Active Parallel Sprint (if any)
Sprint [NUMBER]: [Brief description]
Overlap check: [date confirmed]

## Current Status
[Discovery / Planning / In Progress / Verification / Complete / Blocked]

## Known Blockers
[Blocker or "None"]

## Last Completed Step
[What was the last thing finished?]

## Next Intended Step
[What is the very next action?]

## Files Recently Changed
[filename] - [brief note on what changed]
```

## 10.3 DOMAIN.md

```
# DOMAIN.md Business Logic and Definitions
## Project Overview
[Two to three sentences describing what this project does and why it exists.]

## Core Entities
### [Entity Name]
Definition: [Exact meaning in this project]
Key fields: [Important properties]
Business rules: [How this entity behaves]
What it is NOT: [Clarify common confusions]

## Workflows
### [Workflow Name]
1. [Step 1]
2. [Step 2]
Edge case: [What happens if X fails?]

## Out of Scope
This project explicitly does NOT handle:
[Item 1]
[Item 2]
```

## 10.4 DECISIONS.md (ADR Format)

```
# DECISIONS.md Architecture Decision Ledger
Decisions are never deleted - only superseded by new entries.

## Decision 001: [Short title]
**Date:** [YYYY-MM-DD]
**Status:** [Accepted / Superseded by Decision NUMBER]
**Type:** Architecture / Dependency / Data Model / Security / Other

### Context
[What situation forced this decision?]

### Decision
[What was decided?]

### Tradeoffs
[What this costs or complicates]
```

## 10.5 COMMANDS.md

```
# COMMANDS.md Project Commands
## Environments
| Environment | Base URL | Database | Notes |
|---|---|---|---|
| dev | localhost:[port] | [local DB] | Safe to reset |
| staging | [staging URL] | [staging DB] | QA only |
| production | [prod URL] | [prod DB] | Requires approval |

## Development [dev only]
[command to start dev server]

## Testing [dev and staging]
[command to run all tests]

## Build
[command to build for production]

## Lint / Type Check
[lint command]
```

## 10.6 STYLE_GUIDE.md

```
# STYLE_GUIDE.md Project Conventions
## Code Style
Language: [e.g. TypeScript strict mode]
Formatter: [e.g. Prettier]
Linter: [e.g. ESLint]
```

```
## Naming Conventions
| Type | Convention | Example |
|---|---|---|
| Components | PascalCase | UserProfileCard |
| Functions | camelCase | getUserById |
| Constants | SCREAMING_SNAKE | MAX_RETRY_COUNT |

## What Requires Architect Approval Before Proceeding
- New database tables or schema changes
- New third-party dependencies
- Auth or permission changes
```

## 10.7 SECURITY.md

```
# SECURITY.md Data Security Protocol
## The Absolute Rule
The following must NEVER appear in any file uploaded to an LLM session:
- API keys or tokens of any kind
- Database connection strings or passwords
- Contents of env or .env.* files
- Real user PII: names, emails, phone numbers, addresses, ID numbers

## Redaction Protocol for Mode 4 Discovery
Before uploading any operational data file:
1. Replace all email addresses with user_NNN@example.com
2. Replace all API keys with [API_KEY_REDACTED]
3. Save the redacted version as [filename]_redacted.[ext]
```

## 10.8 GIT_STRATEGY.md

```
# GIT_STRATEGY.md
## Branch Model
- main: production-ready only
- dev: integration branch
- sprint/NNN-short-description: one branch per sprint
- refactor/short-description: used for Mode 3 refactor or migration work

## Rules
1. Builder works only on the assigned branch.
2. Sprint branches merge into dev only after human_review.md passes.
3. dev merges into main only after Architect approval.
4. Mode 1 quick patches may work directly on dev only when under 10 lines and no business logic is introduced.
```

## 10.9 PROMPT_CHANGELOG.md

```
# PROMPT_CHANGELOG.md
## Prompt Version Log
### Version [NUMBER] - [YYYY-MM-DD]
**Prompt affected:** [Prompt name]
```

```
**Change type:** Added / Revised / Deprecated
**Reason:** [Why the change was made]
**Impact:** [How future sessions should behave differently]
```

## 10.10 Sprint: requirements.md

```
# Sprint [NUMBER] Requirements
**Sprint Goal:** [One sentence]
**Status:** Planning / In Progress / Complete
**Branch:** sprint/[NNN]-[description]

## Functional Requirements
1. [Requirement 1]

## Scope Boundaries
### In Scope
- [Feature A]

### Out of Scope (do not build this sprint)
- [Feature X]
```

## 10.11 Sprint: blueprint.md

```
# Sprint [NUMBER] Technical Blueprint
**Architect:** [LLM used]

## Implementation Plan
### Step 1: [Step Name]
What: [Description]
Files affected: [list]
Logic: [How it works]

## Architect Assumptions
- [List every assumption made. Human must verify each before approving.]
```

## 10.12 Sprint: acceptance.md

```
# Sprint [NUMBER] Acceptance Criteria
All items must be checked before this sprint is considered complete.

## Functional Criteria
- [ ] [User can do X]
- [ ] [Feature Y works correctly]

## Technical Criteria
- [ ] No type errors
- [ ] Build completes successfully
- [ ] Test results pasted into implementation_log.md
```

```
## Verification Commands (run in order)
1. [test command]
2. [build command]
```

## 10.13 Sprint: dry_run.md

```
# Sprint [NUMBER] Builder Dry Run
**Date:** [YYYY-MM-DD]
**Builder Tool:** [Tool name]
**Permission Level Requested:** [0-4]

## Files I Intend to Create
| File Path | Change Type | Reason |
|---|---|---|
| src/[path] | Additive | [Why needed] |

## Files I Intend to Modify
| File Path | Change Type | What Changes |
|---|---|---|
| src/[path] | Refactor | [What] |

**STOPPED. Awaiting Architect approval and permission level assignment.**
```

## 10.14 Sprint: implementation_log.md

```
# Sprint [NUMBER] Implementation Log
**Date:** [YYYY-MM-DD]
**Builder Tool:** [Tool name]
**Permission Level Used:** [Level]

## Files Created / Modified
- src/[path]: [Description]

## Verification Results
### Test Output
[Paste terminal output here]
Result: Passed / Failed
```

## 10.15 Sprint: human_review.md

```
# Sprint [NUMBER] Human Approval Gate
## The Five Gates
### Gate 1: Scope
- [ ] implementation_log.md lists only files that appeared in dry_run.md

### Gate 2: Tests
- [ ] Passing test results are present in implementation_log.md

### Gate 3: Debt
```

```
- [ ] All known issues from this sprint are in planning/backlog.md

### Gate 4: Decisions
- [ ] All new dependencies have DECISIONS.md entries

### Gate 5: State
- [ ] STATE.md reflects completed sprint status

## Approved to commit: Yes / No
```

## 10.16 Sprint: retrospective.md

```
# Sprint [NUMBER] Retrospective
## What Went Well / What Broke or Was Unclear
- [Item]

## Blueprint Quality
- Was requirements.md clear? [Yes / No]
- Was blueprint.md accurate? [Yes / No]

## Time Estimate vs Actual
- Estimated: [X hours] / Actual: [Y hours]
```

## 10.17 Sprint: rollback_log.md

```
# Sprint [NUMBER] Rollback Log
## Pre-Implementation (fill in before Builder begins - Level 4 required)
**Current clean commit hash:** [git log --oneline -1]
**Rollback command:** `git checkout [branch] && git revert HEAD~[N] --no-edit`

## Post-Implementation (fill in only if recovery was needed)
**Root cause identified:** [what went wrong]
**State after revert:** [Clean / Partial - describe]
```

## 10.18 planning/backlog.md

```
# Backlog
## Backlog Item 001: [Short title]
**Date added:** [YYYY-MM-DD]
**Type:** Feature / Bug / Refactor / Tech Debt
**Priority:** High / Medium / Low / Unprioritized
**Status:** Unprioritized / Accepted / Scheduled for Sprint [NNN] / Rejected

### Description
[What is this item and why it matters]
```

# 11. Domain Adaptations and Starter Template Library

## 11.1 Domain Adaptations

The framework structure is identical across project types. What changes is the content of the key files.

The framework structure stays the same across project types. What changes is the content of the key files.

### Web Application (Next.js / React)

Entities map routes, API endpoints, and context components. COMMANDS.md manages package scripts such as npm run build and npm run lint.

### Game Development (Unity / Godot)

Entities track scenes, game states, event handlers, and asset flows. DOMAIN.md functions like a technical Game Design Document.

### Desktop Applications (Electron / Tauri)

Requires explicit local filesystem configuration parameters and safe machine-access bounds tables.

### Data Ingestion & Automation (Python ETL)

Relies heavily on Mode 4 data audits and strict input data validation suites.

## 11.2 Starter Template Library

The _templates/ folder stores pre-built project scaffolds. Copy the closest template, run a find-and-replace on placeholders, fill in business constraints, and initialize your kickoff prompt session.

```
_templates/
├── web-app/          # Next.js + TypeScript core defaults
├── game-unity/       # Unity structural asset configurations
├── game-godot/       # Godot script node structures
├── desktop-electron/  # Electron main/renderer processes
└── data-parser/      # Python data handling validation routines

# Ecosystem setup script sequence
cp -r _templates/web-app/* /path/to/[PROJECT_NAME]/
# Then run global workspace find-and-replace for [PROJECT_NAME].
```

# 12. Comprehensive Prompt Catalog

These prompts are model-agnostic and are injected directly into active console or browser sessions as you transition operational modes. The catalog includes the core prompts from the framework and fills in the lifecycle prompts referenced by the process so the document is usable end-to-end.

## 12.1 Persona Injector

You are the System Architect operating within the Autonomous Duck Deployment Framework for [PROJECT_NAME]. Your role is strictly systems engineering, operational discovery, and strategic orchestration. You operate strictly above the code execution layer.

RIGID CONSTRAINTS:

1. Never write application source files directly.
2. Before approving any work, run a structural Discovery Gate. Interrogate assumptions, surface edge cases, and define clear data verification pipelines.
3. Your primary engineering outputs are structured blueprint packages containing explicit sprint requirements, blueprints, and acceptance parameters.
4. Ground your choices strictly within our DOMAIN.md logic boundaries.

## 12.2 Project Kickoff Interview Prompt

Execute Project Kickoff Interview Protocol for [PROJECT_NAME].

I will provide raw notes, ideas, rough goals, constraints, and known unknowns. Your task is to ask up to 10 clarifying questions before creating any final architecture. Prioritize questions that affect business logic, data shape, security, user workflows, deployment, and scope boundaries.

After I answer, generate draft contents for:

1. DOMAIN.md
2. STATE.md
3. STYLE_GUIDE.md
4. docs/architecture.md
5. DECISIONS.md starter ledger
6. QUESTIONS.md starter ledger
7. RISKS.md starter ledger

Do not write application source code. Produce Markdown file blocks only.

## 12.3 Resumption Prompt

Execute Resumption Protocol.

Loaded files:

- STATE.md
- DOMAIN.md
- AGENTS.md

Summarize:

1. Current project goal
2. Active sprint or paused status
3. Known blockers
4. Last completed step
5. Next safest action
6. Files that must be loaded before any Builder session

Do not invent missing state. If a required detail is absent, add it to QUESTIONS.md instead of assuming.

## 12.4 Sprint Pack Generation Prompt

Execute Architect Sprint Generation Protocol.

Active Project State: [Paste current contents of STATE.md]
Target Sprint Scope: Sprint [NUMBER] - [Summarize target feature assignment in one sentence]
Structural References: Refer to our active DOMAIN.md parameters.

Generate a comprehensive blueprint pack for this sprint loop. You must output three distinct Markdown file blocks ready for local file system saving:
1. REQUIREMENTS.MD: functional business parameters, technical constraints, and clear in-scope versus out-of-scope boundaries.
2. BLUEPRINT.MD: step-by-step implementation design tracking specific files to modify or instantiate with explicit change classifications.
3. ACCEPTANCE.MD: checkbox-format validation requirements mapping clean compilation scripts and automated test coverage rules.

Do not output raw application source code. Provide only blueprint specification parameters.

## 12.5 Builder Pre-Flight Dry Run Prompt

Initialize Pre-Flight Dry Run Protocol.

Your current allocated Permission Level for this session: Level 0 (Read-Only).

Active Task Specifications:
- Refer to planning/sprints/sprint_[NUMBER]/requirements.md
- Refer to planning/sprints/sprint_[NUMBER]/blueprint.md

Prior to creating or mutating any file inside the /src directory, execute a read-only analysis of the repository and return a Pre-Flight Summary sheet.

YOUR REQUIRED CHECKLIST:
1. Files to Create: Specify paths and verify intention is Additive.
2. Files to Modify: Specify paths, classify change type (Refactor/Destructive), and provide technical rationale.
3. Out of Scope: Identify files or features explicitly marked out of scope and confirm you will leave them untouched.

CRITICAL INSTRUCTION: Output your report using the standard dry_run.md file format and then STOP immediately. Do not generate source file scripts until you receive explicit operator clearance.

## 12.6 Architect Sanity Audit Prompt

Execute Architect Sanity Audit Protocol.

Inputs:
- planning/sprints/sprint_[NUMBER]/requirements.md
- planning/sprints/sprint_[NUMBER]/blueprint.md

- planning/sprints/sprint_[NUMBER]/acceptance.md
- planning/sprints/sprint_[NUMBER]/dry_run.md

Check the dry run for:
1. Hidden scope bleed
2. Files not approved by blueprint.md
3. Business logic not defined in DOMAIN.md
4. Missing verification commands
5. Dependency additions that require DECISIONS.md entries
6. Security or secrets exposure risk

Return one of these outcomes:
- APPROVED: Permission Level [1-4] authorized, with reason.
- REJECTED: explain required changes to blueprint_feedback.md.

Do not write source code.

## 12.7 Builder Implementation Authorization Prompt

Dry run verified by Architect. Permission Level [LEVEL] authorized.

Proceed with implementation according to:
- planning/sprints/sprint_[NUMBER]/requirements.md
- planning/sprints/sprint_[NUMBER]/blueprint.md
- planning/sprints/sprint_[NUMBER]/acceptance.md

Rules:
1. Modify only the approved files.
2. Do not invent new business rules.
3. Log every created or modified file in implementation_log.md.
4. Run all verification commands listed in acceptance.md and COMMANDS.md.
5. Paste raw terminal output into implementation_log.md.
6. Stop if a blocking ambiguity appears.

## 12.8 Architect Implementation Review Prompt

Execute Implementation Review Protocol.

Inputs:
- acceptance.md
- dry_run.md
- implementation_log.md
- test output

Review whether the implementation satisfies the sprint acceptance criteria. Verify that implementation_log.md lists only files that appeared in dry_run.md unless an approved expansion was granted. Identify any missing tests, unresolved debt, scope creep, or required DECISIONS.md entries.

Return:

1. Approved / Not Approved
2. Required fixes before commit
3. Required updates to STATE.md, DECISIONS.md, RISKS.md, or backlog.md

### 12.9 Cross-Sprint Consistency Audit Prompt

Execute Cross-Sprint Consistency Audit.

Inputs:
- DOMAIN.md
- STATE.md
- DECISIONS.md
- Current and recent sprint folders
- docs/data_model.md and docs/api.md if applicable

Check for drift between what the blueprints built and what DOMAIN.md says. Identify invented entities, inconsistent field names, outdated docs, unresolved risks, missing decision records, and backlog items that should become sprint candidates.

Return:
1. Consistency score: Green / Yellow / Red
2. Drift findings
3. Required updates before next sprint
4. Recommended next sprint candidates

## 13. Quick Reference Checklists

### 13.1 Before the first AI session

- SECURITY.md has a safe-to-share and never-share list.
- DOMAIN.md has project overview, core entities, workflows, and out-of-scope boundaries.
- COMMANDS.md has development, test, build, lint/type-check, deploy, and rollback commands.
- STATE.md says exactly what step you are on.
- QUESTIONS.md has no blocker that prevents Sprint 001 planning.

### 13.2 Before Builder writes code

- Sprint branch exists.
- requirements.md, blueprint.md, and acceptance.md exist.
- Builder has completed Level 0 dry run.
- Architect has approved dry_run.md and assigned permission level.
- New dependencies have DECISIONS.md entries.
- No secrets or PII are loaded into the session.

### 13.3 Before committing a sprint

- implementation_log.md lists created and modified files.
- Test/build/lint output is pasted into implementation_log.md.

- human_review.md passes all five gates: Scope, Tests, Debt, Decisions, State.
- STATE.md has been updated.
- retrospective.md records what worked, broke, and should change next time.
- Builder thread has been closed after the work is captured in files.

# 14. Glossary

| Term | Meaning |
|---|---|
| Acceptance Criteria | The checklist that defines whether a sprint is complete. |
| ADR | Architecture Decision Record. A permanent written record of why a technical decision was made. |
| Architect Mode | The planning mode. It defines requirements, risks, blueprints, and acceptance criteria without writing source code. |
| Builder Mode | The implementation mode. It writes code only after a dry run and explicit permission. |
| Context Bleed | When old or unrelated chat context contaminates the current task. |
| Context Engineering | The practice of organizing project knowledge into clean, specific files rather than relying on long chat history. |
| Dry Run | A read-only plan from the Builder describing intended file changes before code mutation. |
| Harness-First Development | Writing or defining the test harness before scaling functional implementation. |
| Local File Sovereignty | The principle that project memory belongs in local files controlled by the human operator, not in an AI chat session. |
| Mode | A task category such as Quick Patch, Feature Sprint, Refactor/Migration, Data Ingestion, or Style Audit. |
| Permission Level | A numbered limit on what the Builder may do, from Level 0 read-only to Level 4 migration. |
| Sprint Pack | The three-file package for a sprint: requirements.md, blueprint.md, and acceptance.md. |
| Sovereign Brain | The file system folder containing the Markdown files that define the project state and rules. |

| | |
|---|---|
| State Reset | Closing the active Builder chat after the work is recorded, so the next sprint begins with clean context. |
| Vibe Coding | An informal AI coding loop where the user keeps prompting in one chat without stable project memory or explicit specs. |